

Manual do SDK de Física Bullet 2.83



Também confira os fóruns e a wiki em bulletphysics.org

© Erwin Coumans 2015
Todos os direitos reservados.

Índice

MANUAL DO SDK DE FÍSICA DO BULLET 2.83.....	1
1 Introdução	4Descrição
da biblioteca.....	4
Principais Recursos.....	
4Contact e Suporte	Sistema de
construção 42, como começar e o que há de novo	5
Por favor, veja um guia separado de BulletQuickstart.pdf.	53
Visão Geral da Biblioteca	6
Introdução.....	6
.....	de Design de Software
6Pipeline de Física de Corpos Rígidos.....	
7Visão geral da integração	7Tipos
básicos de dados e biblioteca matemática	
9Gerenciamento de Memória, Alinhamento, Containers.....	
9Temporização e Perfil de Desempenho.....	
10Debug Desenho	114
Detecção de Colisão de Balas	12 Detecção de
Colisão	12
Formas de colisão.....	13Primitivas
Convexas	13Formas
compostas.....	14Formas Convexas
de Invólucro	14 Malhas
triangulares côncavas.....	14Decomposição
Convexa.....	14Altura do campo
.....	
15btEstáticoFormaPlano.....	15Escala
de Formas de Colisão.....	Margem de colisão
de 15.....	15Matriz de
Colisão.....	16Registrar formas e
algoritmos personalizados de colisão.....	165 Filtragem de Colisões
(colisões seletivas)	17
Filtrando colisões usando máscaras.....	
17Filtragem de colisões usando um callback de filtro de fase larga	
17Filtrar colisões usando um NearCallback personalizado.....	
18Derivando sua própria classe do btCollisionDispatcher.....	186
Dinâmica de Corpo Rígido.....	19
Introdução.....	19Corpos
rígidos estáticos, dinâmicos e cinemáticos.....	19Centro de
massa Transformação Mundial	20O que é um
Estado de Movimento?.....	
20Interpolação.....	21Então,
como uso um?.....	
21DefaultMotionState	21Corpos
Cinemáticos	22Quadros de
simulação e quadros de interpolação	227
Restrições.....	23
Restrição de Ponto a Ponto	Restrição
da dobradiça 23.....	23Restrição do
Deslizante	Restrição de Torção
24Cone	24Generic 6 Restrição de
Dof	248 Ações: Veículos e Controlador
de Personagem	26

Interface de Ação	Veículo
26Raycast.....	Controlador de 26
caracteres.....	269 Dinâmica de Corpo
Mole	27
Introdução.....	27Construção a
partir de uma malha triangular.....	27Clusters de
colisão.....	27Aplicando forças a
um corpo mole.....	28Restrições de corpo
mole.....	Exemplo de Bullet 2810
Navegador	29
BSP Demo	Demonstração de 30
veículos	Demonstração do Empilhadeiro
30	3011 Demonstrações Técnicas
Avançadas de Baixo Nível	31
Demonstração de Interface de Colisão	Demonstração
de 31Collision.....	31Algoritmo de Colisão do
Usuário	Demonstração Convexa de Elenco /
Varredura 31Gjk	31 Colisão Convexa Contínua
.....	Demonstração do
31Raytracer.....	Demo
31Simplex.....	3212 Ferramentas de Autoria e
Serialização	33
Plugin Dynamica Maya	33Blender
.....	33Cinema 4D,
Lightwave CORE, Houdini.....	33Serialização e o
formato binário Bullet .bullet	3413 Dicas Gerais
.....	35
Evite formas de colisão muito pequenas e muito grandes.....	
35Evite grandes razões de massa (diferenças)	
35Combine múltiplas malhas triangulares estáticas em uma só.....	
35Utilize o intervalo de tempo fixo interno padrão.....	
35Para ragdolls use btConeTwistConstraint	35Não
defina a margem de colisão para zero	35Usar
menos de 100 vértices em uma malha convexa	36Evite
triângulos enormes ou degenerados em uma malha triangular.....	36O
recurso de perfilamento btQuickProf contorna o alocador de memória	36Por
valor de atrito e restituição do triângulo	37Outros
solucionadores de restrições MLCP	Modelo
de atrito personalizado	Paralelismo
3714 usando OpenCL.....	38
OpenCL corpo rígido e detecção de colisão.	
3815 Documentação adicional e referências	39
Recursos online.....	
39Ferramentas de Autoria	
39Livros	
39Contribuições e pessoas.....	40

1 Introdução

Descrição da biblioteca

Bullet Physics é uma biblioteca profissional de código aberto para detecção de colisão, corpo rígido e dinâmica de corpo mole, escrita em C++ portátil. A biblioteca é projetada principalmente para uso em jogos, efeitos visuais e simulação robótica. A biblioteca é gratuita para uso comercial sob a licença ZLib.

Principais Características

- Detecção de colisão discreta e contínua, incluindo teste de varredura de raios e convexos. As formas de colisão incluem malhas côncavas e convexas e todas as primitivas básicas
- Coordenadas maximais de 6 graus de liberdade de corpos rígidos (btRigidBody) conectadas por restrições (btTypedConstraint), bem como multi-corpos coordenadas generalizadas (btMultiBody) conectados por mobilizadores usando o algoritmo do corpo articulado.
- Solucionador rápido e estável de dinâmicas de dinâmica de corpo rígido, dinâmica de veículos, controlador de caracteres e slider, dobradiça, 6DOF genérico e restrição de torção de cone para ragdolls
- Dinâmica de corpos macios para tecido, corda e volumes deformáveis com interação bidirecional com corpos rígidos, incluindo suporte de restrições
- Código C++ open source sob licença Zlib e gratuito para uso comercial em todas as plataformas, incluindo PLAYSTATION 3, Xbox 360, Wii, PC, Linux, Mac OSX, Android e iPhone
- Plugin Maya Dynamica, integração com Blender, serialização binária nativa .bullet e exemplos de como importar arquivos URDF, Wavefront .obj e Quake .bsp.
- Muitos exemplos mostrando como usar o SDK. Todos os exemplos são fáceis de navegar no navegador de exemplos OpenGL 3. Cada exemplo também pode ser compilado sem gráficos.
- Guia Quickstart, documentação do Doxygen, wiki e fórum complementam os exemplos.

Contato e Suporte

- Fórum público para apoio e feedback está disponível em <http://bulletphysics.org>

2 Sistema de construção, como começar e novidades

Por favor, veja um guia separado de `BulletQuickstart.pdf`.

A partir do Bullet 2.83 há um guia de início rápido separado. Este guia de início rápido inclui as mudanças e novos recursos das novas versões do SDK, além de como construir o SDK BulletPhysics e os exemplos.

Você pode encontrar esse guia de início rápido em `Bullet/docs/BulletQuickstart.pdf`.

3 Visão Geral da Biblioteca

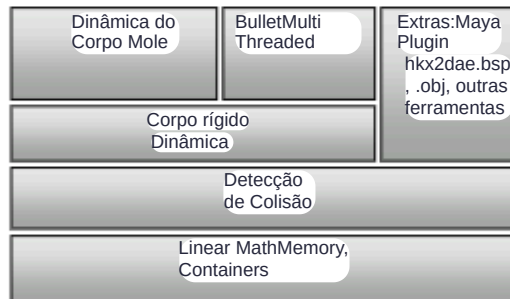
Introdução

A principal tarefa de um motor de física é realizar a detecção de colisão, resolver colisões e outras restrições, e fornecer a transformação de mundo atualizada¹ para todos os objetos. Este capítulo dará uma visão geral do pipeline de dinâmica de corpos rígidos, bem como dos tipos básicos de dados e da biblioteca matemática compartilhada por todos os componentes.

Software Design

Bullet foi projetado para ser personalizável e modular. O desenvolvedor pode

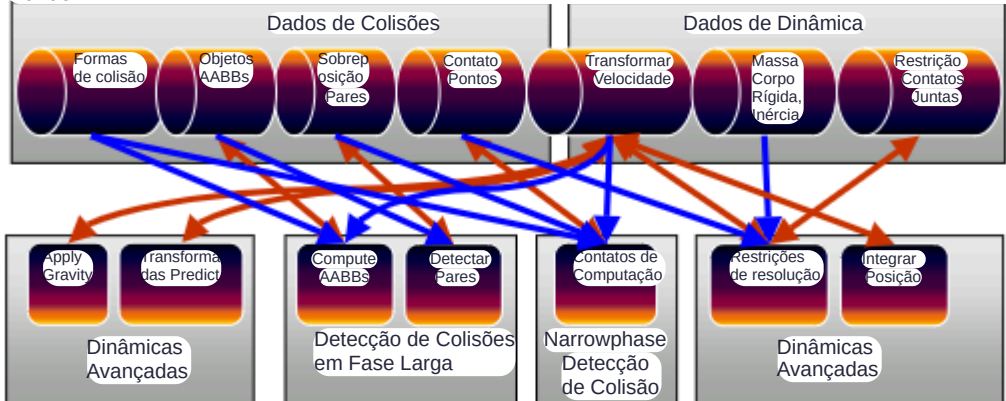
- usar apenas o componente de detecção de colisão
- usar o componente de dinâmica de corpos rígidos sem o componente de dinâmica de corpos moles
- usar apenas pequenas partes da biblioteca e expandi-la de várias maneiras
- escolher usar uma versão de precisão simples ou dupla da biblioteca
- usar um alocador de memória personalizado, conectar o próprio perfil de desempenho ou gaveta de depuração Os principais componentes são organizados da seguinte forma:



¹ Transformada de mundo do centro de massa para corpos rígidos, vértices transformados para corpos moles

Pipeline de Física de Corpos Rígidos

Antes de entrar em detalhes, o diagrama a seguir mostra as estruturas de dados e estágios de computação mais importantes no pipeline de física Bullet. Esse pipeline é executado da esquerda para a direita, começando aplicando a gravidade e terminando pela integração de posição, atualizando a transformação do mundo.



Toda a computação do pipeline de física e suas estruturas de dados são representadas no Bullet pelo mundo `adynamic`. Ao realizar a 'simulação por passos' no mundo da dinâmica, todos os estágios acima são executados. A implementação padrão do mundo dinâmico é o `btDiscreteDynamicsWorld`. Bullet permite que o desenvolvedor escolha explicitamente várias partes do mundo dinâmico, como detecção de colisão em `broadphase`, detecção de colisão em fase estreita (`dispatcher`) e solucionador de restrições.

Visão geral da integração

Se você quiser usar o Bullet em seu próprio aplicativo 3D, o melhor é seguir os passos da demonstração `HelloWorld`, localizada em `Bullet/examples/HelloWorld`. Em poucas palavras:

- Criar um `btDiscreteDynamicsWorld` ou `btSoftRigidDynamicsWorld`

Essas classes, derivadas do `btDynamicsWorld`, fornecem uma interface de alto nível que gerencia objetos e restrições de seu físico. Também implementa a atualização de todos os objetos a cada quadro.

- Criar um `btRigidBody` e adicioná-lo ao `btDynamicsWorld`

Para construir um `btRigidBody` ou `btCollisionObject`, você precisa fornecer:

- Massa, positiva para objetos móveis dinâmicos e 0 para objetos estáticos

- Forma de colisão, como uma caixa, esfera, cone, invólucro convexo ou malha triangular
- Propriedades materiais como atrito e restauração

Atualize a simulação a cada quadro:

- `stepSimulation`

Chame o `stepSimulation` do mundo da dinâmica. O `btDiscreteDynamicsWorld` leva automaticamente em conta o passo de tempo variável ao realizar interpolação em vez de simulação para pequenos passos de tempo. Ele utiliza um intervalo de tempo fixo interno de 60 Hertz. `stepSimulation` realizará detecção de colisão e simulação física. Ele atualiza o world transform para objetivos ativos chamando de `setWorldTransform` do `thebtMotionState`.

Os próximos capítulos fornecerão mais informações sobre detecção de colisões e dinâmica de corpo rígido. Muitos dos detalhes são demonstrados no Bullet/exemplos. Se você não encontrar certas funcionalidades, por favor, visite o fórum de física no site do Bullet em <http://bulletphysics.org>

Tipos Básicos de Dados e Biblioteca Matemática

Os tipos básicos de dados, gerenciamento de memória e contêineres estão localizados em `Bullet/src/LinearMath`.

- `btScalar` `btScalar` é uma palavra sofisticada para um número de ponto flutuante. Para permitir a compilação da biblioteca em precisão de ponto flutuante simples e precisão dupla, usamos o tipo de dado `btScalar` em toda a biblioteca. Por padrão, `btScalar` é um `typedef` para `float`. Pode ser dobrado por definindo `BT_USE_DOUBLE_PRECISION` seja no seu sistema de build, ou no topo do arquivo `Bullet/src/LinearMath/btScalar.h`.
- as posições e vetores do `btVector3` podem ser representados usando o `btVector3`. `btVector3` possui 3 componentes escalares `x`, `y`, `z`. No entanto, ele tem um quarto componente `w` não utilizado por motivos de alinhamento e compatibilidade com SIMD. Muitas operações podem ser realizadas em um `btVector3`, como somar e subtrair e tomar o comprimento de um vetor.
- as orientações e rotações de `btQuaternion` e `btMatrix3x3` podem ser representadas usando tanto `btQuaternion` quanto `btMatrix3x3`.
- `btTransform` `btTransform` é uma combinação de posição e orientação. Pode ser usado para transformar pontos e vetores de um espaço de coordenadas para outro. Não é permitido descamar ou aparar. Bullet utiliza um sistema de coordenadas para destros:

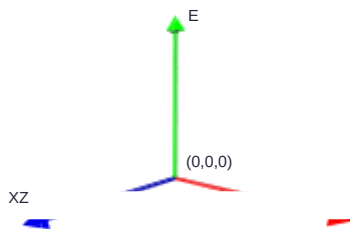


Figura 1 Sistema de coordenadas para destros

`btTransformUtil` e `btAabbUtil` fornecem funções utilitárias comuns para transformações e AABBs.

Gerenciamento de Memória, Alinhamento, Contêineres

Frequentemente, é importante que os dados estejam alinhados a 16 bytes, por exemplo, ao usar transferências SIMD ou DMA em SPU Cell. O Bullet fornece alocadores de memória padrão que gerenciam o alinhamento, e os desenvolvedores podem fornecer seu próprio alocador de memória. Todas as alocações de memória no uso do Bullet:

- `btAlignedAlloc`, que permite especificar tamanho e alinhamento
- `btAlignedFree`, liberar a memória alocada pelo `btAlignedAlloc`.

Para sobrescrever o alocador de memória padrão, você pode escolher entre:

- `btAlignedAllocSetCustom` é usado quando seu alocador personalizado não suporta alinhamento
- `btAlignedAllocSetCustomAligned` pode ser usado para definir seu alocador de memória personalizado alinhado. Para garantir que uma estrutura ou classe será alinhada automaticamente, você pode usar esta macro:
 - `ATTRIBUTE_ALIGNED16(type) variablename` cria uma variável alinhada de 16 bytes
- Frequentemente, é necessário manter um array de objetos. Originalmente, a biblioteca Bullet usava uma estrutura STL `std::vector<data>` para arrays, mas por razões de portabilidade e compatibilidade mudamos para nossa própria classe de array.
- `btAlignedObjectArray` se assemelha muito a `std::vector`. Ele utiliza o alocador alinhado para garantir o alinhamento. Ele possui métodos para ordenar o array usando quick sort ou heap sort. Para permitir que o Microsoft Visual Studio Debugger visualize `btAlignedObjectArray` e `btVector3`, siga as instruções em `Bullet/msvc/autoexp_ext.txt`

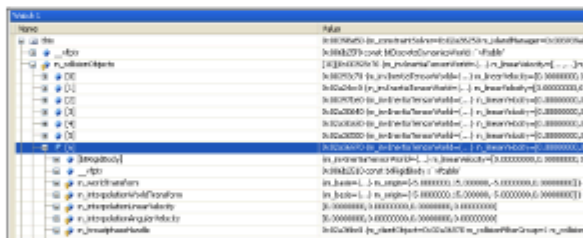


Figura 2 Visualização de Depuração MSVC

Temporização e Perfil de Desempenho

Para localizar gargalos no desempenho, o Bullet utiliza macros para medição hierárquica de desempenho.

- o `btClock` mede o tempo usando precisão em microssegundos.
- `BT_PROFILE(section_name)` marca o início de uma seção de perfilamento.

- `CProfileManager::dumpAll()`; Faz dump de uma saída de desempenho hierárquica no `TheConsole`. Chame isso depois de passar pela simulação.

• `CProfileIterator` é uma classe que permite iterar pela árvore de perfilamento. Note que o profiler não usa o alocador de memória, então talvez seja bom desabilitá-lo ao verificar vazamentos de memória ou ao criar uma versão final do seu software. O recurso de perfilamento pode ser desativado definindo `#define BT_NO_PROFILE 1` em `Bullet/src/LinearMath/btQuickProf.h`

Desenho de Depuração

Depuração visual das estruturas de dados da simulação pode ser útil. Por exemplo, isso permite verificar se os dados de simulação física correspondem aos dados gráficos. Também aparecem problemas de escalonamento, restrições de frames ruins e limites. `btIDebugDraw` é a classe de interface usada para desenhos de depuração. Derive sua própria classe e implemente a `'drawLine'` virtual e outros métodos. Atribua seu gaveta de depuração personalizado ao mundo das dinâmicas usando o método `setDebugDrawer`. Depois, você pode escolher desenhar recursos específicos de depuração definindo o modo da gaveta de depuração:

```
dynamicsWorld->getDebugDrawer()->setDebugMode(debugMode); A  
cada frame você pode chamar o desenho de depuração chamando o  
mundo-> debugDrawWorld(); 2Aqui  
estão alguns modos de depuração
```

- `btIDebugDraw::D_BG_DrawWireframe`
- `btIDebugDraw::D_BG_DrawAabb`
- `btIDebugDraw::D_BG_DrawConstraints`
- `btIDebugDraw::D_BG_DrawConstraintLimits` Por padrão, todos os objetos são visualizados para um dado modo de depuração, e ao usar muitos objetos isso atrapalha a exibição. Você pode desativar o desenho de depuração para objetos específicos usando

```
int f = objects->getCollisionFlags();  
ob->setCollisionFlags(f|btCollisionObject::CF_DISABLE_VISUALIZE_OBJECT);
```

2 Este recurso é suportado tanto para `btCollisionWorld` quanto para `btDiscreteDynamicsWorld`

4 Detecção de Colisão de Balas

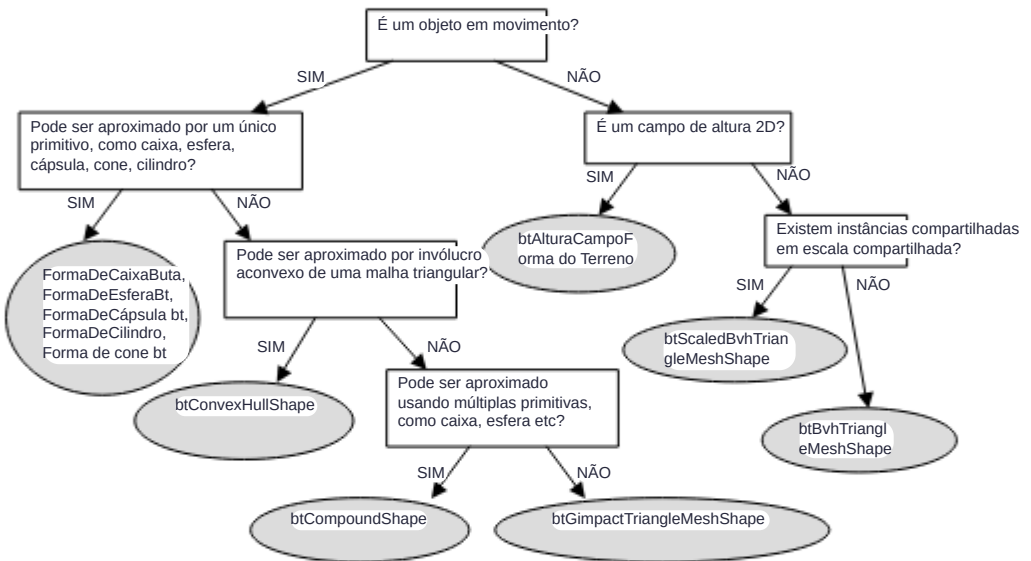
Detecção de Colisão

A detecção de colisão fornece algoritmos e estruturas de aceleração para consultas de ponto mais próximo (distância e penetração), bem como testes de varrimento de raios e convexos. As principais estruturas de dados são:

- `btCollisionObject` é o objeto que possui uma transformação de mundo e uma forma de colisão.
- `btCollisionShape` descreve a forma de colisão de um objeto de colisão, como caixa, esfera, casco convexo ou malha triangular. Uma única forma de colisão pode ser compartilhada entre objetos de múltiplas colisões.
- `btGhostObject` é um `btCollisionObject` especial, útil para consultas de colisão localizadas rápidas.
- `btCollisionWorld` armazena todos os `btCollisionObjects` e fornece uma interface para realizar consultas. A detecção de colisão em fase larga fornece uma estrutura de aceleração para rejeitar rapidamente pares de objetos com base na sobreposição da caixa delimitadora alinhada ao eixo (AABB). Existem várias estruturas de aceleração de fase larga disponíveis:
 - `btDbvtBroadphase` usa uma hierarquia dinâmica rápida de volumes delimitadora baseada na árvore AABB
 - `btAxisSweep3` e `bt32BitAxisSweep3` implementam varredura e poda incremental 3D
- `btSimpleBroadphase` é uma implementação de referência de força bruta. É lento, mas fácil de entender e útil para depurar e testar uma fase mais avançada em broadphase. A broadphase adiciona e remove pares sobrepostos de um cache de pares. Pares sobrepostos são persistentes ao longo do tempo e podem armazenar em cache informações como forças de restrição de contato anterior que podem ser usadas para o 'warmstarting': usando a solução anterior para convergir mais rapidamente para a resolução de restrições. Um despachante de colisão faz itera em cada par, busca um algoritmo de colisão correspondente baseado nos tipos de objetos envolvidos e executa o algoritmo de colisão calculando pontos de contato.
 - `btPersistentManifold` é um cache de pontos de contato para armazenar pontos de contato para um determinado par de objetos.

Formas de colisão

Bullet suporta uma grande variedade de formas de colisão diferentes, e é possível adicionar as suas próprias. Para melhor desempenho e qualidade, é importante escolher o formato de colisão que se adequa ao seu objetivo. O diagrama a seguir pode ajudar a tomar uma decisão:



Primitivas Convexas

A maioria das formas primitivas é centrada na origem de seu referencial de coordenadas local:

- btBoxShape : Caixa definida pelas metades extensões (metade do comprimento) de seus lados
- btForma da Esfera : Esfera definida por seu raio
- CápsulaForma: Cápsula ao redor do eixo Y. Também
- btCapsuleShapeX/ZbtCylinderShape : Cilindro ao redor do eixo Y. Também
- btCylinderShapeX/Z.btConeShape : Cone ao redor do eixo Y. Também
- btConeShapeX/Z.

btMultiSphereShape : Invólucro convexo de múltiplas esferas, que pode ser usado para criar uma cápsula (contornando 2 esferas) ou outras formas convexas.

Formas compostas

Múltiplas formas convexas podem ser combinadas em uma forma composta ou composta, usando `btCompoundShape`. Essa é uma forma côncava feita de subpartes convexas, chamadas formas filha. A forma de cada filho tem sua própria transformada de deslocamento local, em relação à `btCompoundShape`. É uma boa ideia aproximar formas côncavas usando uma coleção de invólucros convexas e armazená-las em `btCompoundShape`. Você pode ajustar o centro de massa usando um método utilitário `btCompoundShape::calculatePrincipalAxisTransform`.

Formas de Invólucro Convexas

Bullet suporta várias formas de representar malhas triangulares convexas. A maneira mais fácil é criar `btConvexHullShape` e passar um array de vértices. Em alguns casos, a malha gráfica contém vértices demais para ser usada diretamente como `btConvexHullShape`. Nesse caso, tente reduzir o número de vértices.

Malhas triangulares côncavas

Para ambientes de mundo estáticos, uma forma muito eficiente de representar malhas triangulares estáticas é usar `btBvhTriangleMeshShape`. Essa forma de colisão constrói uma estrutura interna de aceleração a partir de `btTriangleMesh` ou `btStridingMeshInterface`. Em vez de construir a árvore em tempo de execução, também é possível serializar a árvore binária para disco. Veja `exemplos/ConcaveDemo` de como salvar e carregar esta estrutura de aceleração de árvore `btOptimizedBvh`. Quando você tem várias instâncias da mesma malha triangular, mas com escalonamento diferente, pode instanciar uma forma de malha `btBvhTriangleMeshShape` várias vezes usando a forma `btScaledBvhTriangleMeshShape`. O `btBvhTriangleMeshShape` pode armazenar múltiplas partes de malha. Ele mantém um índice triangular e um índice de parte em uma estrutura de 32 bits, reservando 10 bits para o ID da peça e os 22 bits restantes para o índice triangular. Se precisar de mais de 2 milhões de triângulos, divida a malha triangular em múltiplas submalhas, ou mude o padrão em `#define MAX_NUM_PARTS_IN_BITS` nos arquivos `src/BulletCollision/BroadphaseCollision/btQuantizedBvh.h`.

Decomposição Convexa

Idealmente, malhas côncavas deveriam ser usadas apenas para arte estática. Caso contrário, seu invólucro convexo deve ser usado passando a malha para `btConvexHullShape`. Se uma única forma convexa não for detalhada o suficiente, múltiplas partes convexas podem ser combinadas em um objeto composto chamado `btCompoundShape`. A decomposição convexa pode ser usada para decompor a malha côncava em várias partes convexas. Veja `theDemos/ConvexDecompositionDemo` para uma forma automática de fazer decomposição convexa.

Campo de altura

Bullet oferece suporte para o caso especial de um terreno plano 2D côncavo através do `btHeightfieldTerrainShape`. Veja exemplos/`TerrainDemo` para seu uso.

btStaticPlaneShape

Como o nome sugere, o `btStaticPlaneShape` pode representar um plano infinito ou meio espaço. Essa forma só pode ser usada para objetos estáticos e imóveis. Essa forma foi introduzida principalmente para fins de demonstração.

Escala de Formas de Colisão

Algumas formas de colisão podem ter escala local aplicada. Use `btCollisionShape::setScaling(vector3)`. Escalonamento não uniforme com diferentes valores de escala para cada eixo pode ser usado para `btBoxShape`, `btMultiSphereShape`, `btConvexShape`, `btTriangleMeshShape`. Escala uniforme, usando valor `x` para todos os eixos, pode ser usada para `btSphereShape`. Note que uma esfera escalonada não uniforme pode ser criada usando `btMultiSphereShape` com 1 esfera. Como mencionado antes, `btScaledBvhTriangleMeshShape` permite instanciar uma `btBvhTriangleMeshShape` em diferentes fatores de escala não uniformes. O `btUniformScalingShape` permite instanciar formas convexas em diferentes escalas, reduzindo a quantidade de memória.

Margem de Colisão

Bullet utiliza uma pequena margem de colisão para formas de colisão, a fim de melhorar o desempenho e a confiabilidade da detecção de colisão. É melhor não modificar a margem de colisão padrão e, se você usar um valor positivo: margem zero pode causar problemas. Por padrão, essa margem de colisão é definida para 0,04, o que é 4 centímetros se suas unidades estiverem em medidores (recomendado). Dependendo de quais formas de colisão, a margem tem significados diferentes. Geralmente, a margem de colisão expandirá o objeto. Isso vai criar uma pequena lacuna. Para compensar isso, algumas formas vão subtrair a margem do tamanho real. Por exemplo, o `btBoxShape` subtrai a margem de colisão das metades extensões. Para um `btSphereShape`, todo o raio é margem de colisão, então não haverá espaço. Não substitua a margem de colisão para esferas. Para envoltórios, cilindros e cones convexas, a margem é adicionada às extensões do objeto, então uma lacuna ocorrerá, a menos que você ajuste o tamanho da colisão do meshor gráfico. Para objetos convexas do invólucro, existe um método para remover o espaço introduzido pela margem, encolhendo o objeto. Veja os exemplos/`Importers/ImportBsp` para esse uso avançado.

Matriz de Colisões

Para cada par de tipos de forma, o Bullet despachará um determinado algoritmo de colisão, usando o despachante. Por padrão, toda a matriz é preenchida com os seguintes algoritmos. Note que Convexo representa poliedros, cilindros, cones e cápsulas e outros primitivos compatíveis com GJK. GJK significa Gilbert, Johnson e Keerthi, as pessoas por trás desse algoritmo de cálculo de distâncias convexas. Ele é combinado com a EPA para cálculo da profundidade de penetração. EPA significa Expanding PolytopeAlgorithm, de Gino van den Bergen. Bullet tem sua própria implementação gratuita do GJK e EPA.

	Caixa	Esfera	convexo, cone cilíndrico, cápsula	Composto	Malha triangular
Caixa	boxbox	spherebox	GJK	Composto	concaveconvexo
Esfera	spherebox	esfera	GJK	Composto	concaveconvexo
convexo, cilindro, cone, cápsula	GJK	GJK	gjk ou SAT	Composto	concaveconvexo
Composto	Composto	Composto	Composto	Composto	Composto
Malha triangular	concaveconvexo	concaveconvexo	concaveconvexo	Composto	Gimpact

Registro de formas de colisão personalizadas e algoritmos

O usuário pode registrar um algoritmo personalizado de detecção de colisão e sobrescrever qualquer entrada nessa CollisionMatrix usando o algoritmo `btDispatcher::registerCollisionAlgorithm`. Veja exemplos/`UserCollisionAlgorithm` para um exemplo, que registra um algoritmo de colisão `SphereSphere`.

5 Filtragem de Colisão (colisões seletivas)

Bullet oferece três maneiras simples de garantir que apenas certos objetos colidam entre si: máscaras, callbacks de filtro de fase larga e quasecallbacks. Vale notar que a seleção de colisão baseada em máscara acontece muito mais acima na cadeia de ferramentas do que o callback. Resumindo, se máscaras são suficientes para seus propósitos, use-as; Eles têm melhor desempenho e são muito mais fáceis de usar. Claro, não tente encaixar algo em um sistema de seleção baseado em máscaras que claramente não se encaixa ali só porque o desempenho pode ser um pouco melhor.

Filtragem de colisões usando máscaras

Bullet suporta máscaras bit a bit como forma de decidir se as coisas devem colidir com outras coisas ou receber colisões.

```
int myGroup = 1;

int collideMask = 4;

mundo->addCollisionObject(objecto, meuGroup, MáscaraColisão);
```

Durante a detecção de colisão em fase larga, pares sobrepostos são adicionados a um cache de pares somente quando o themask corresponde ao grupo dos outros objetos (em needsBroadphaseCollision)

```
bool colide = (proxi0->m_collisionFilterGroup & proxi1->m_collisionFilterMask) != 0;

colide = colide && (proxi1->m_collisionFilterGroup & proxy 0->m_collisionFilterMask);
```

Se você tiver mais tipos de objetos além dos 32 bits disponíveis nas máscaras, ou algumas colisões forem ativadas ou desativadas com base em outros fatores, então existem várias formas de registrar callbacks para que implemente lógica personalizada e apenas repasse colisões que são as que você deseja:

Filtragem de colisões usando um callback de filtro de fase larga

Uma forma eficiente é registrar um callback de filtro em fase larga. Esse callback é chamado em um estágio muito inicial do pipeline de colisão e impede a geração de pares de colisão.

```
struct YourOwnFilterCallback : public
btOverlapFilterCallback{

    return true quando pares precisam de colisão virtual bool needBroadphaseCollision(btBroadphaseProxy*
    proxi0, btBroadphaseProxy* proxi1) const{

        bool colide = (proxi0->m_collisionFilterGroup & proxi1->m_collisionFilterMask) != 0;
        colide = colide && (proxi1->m_collisionFilterGroup & proxy 0->m_collisionFilterMask);

        adicionar alguma lógica adicional aqui que colidisse de
        retorno de 'colidimentos' modificado;

    }
};
```

E então criar um objeto dessa classe e registrar esse callback usando:

```
btOverlapFilterCallback * filterCallback = novo YourOwnFilterCallback();
dynamicsWorld->getPairCache()->setOverlapFilterCallback(filterCallback);
```

Filtrando colisões usando um NearCallback personalizado

Outro callback pode ser registrado durante a fase estreita, quando todos os pares são gerados pela broadphase. O `btCollisionDispatcher::dispatchAllCollisionPairs` chama este nearcallback de fase estreita para cada par que passa no teste '`btCollisionDispatcher::needsCollision`'. Você pode personalizar esse nearcallback:

```
void MyNearCallback(btBroadphasePair& collisionPair,
    btCollisionDispatcher& dispatcher, btDispatcherInfo& dispatchInfo) {

    Faça sua lógica de colisão aqui// Só despache as informações de colisão Bullet se quiser
    que a física continue. dispatcher.defaultNearCallback(collisionPair, dispatcher,
    dispatchInfo);}

mDespachante->setarNearCallback(MinHARetorno);
```

Derivando sua própria classe a partir do btCollisionDispatcher

Para um controle ainda mais detalhado sobre o despacho de colisão, você pode derivar sua própria classe `frombtCollisionDispatcher` e sobrescrever um ou mais dos seguintes métodos:

```
    Bool virtual      necessitaColisão(btCollisionObject* body0, btCollisionObject* body1);

    Bool virtual      needsResponse(btCollisionObject* body0, btCollisionObject* body1);

    Void virtual      dispatchAllCollisionPairs(btOverlappingPairCache* pairCache, const
    btDispatcherInfo& dispatchInfo, btDispatcher* dispatcher) ;
```

6 Dinâmica de Corpos Rígidos

Introdução

A dinâmica do corpo rígido é implementada sobre o módulo de detecção de colisão. Ele adiciona forças, massa, inércia, velocidade e restrições.

- `btRigidBody` é usado para simular objetos únicos em movimento a 6 graus de liberdade. `btRigidBody` é derivado de `btCollisionObject`, então herda sua transformação do mundo, atrito e restituição, e adiciona velocidade linear e angular.
- `btTypedConstraint` é a classe base para restrições de corpo rígido, incluindo `btHingeConstraint`, `btPoint2PointConstraint`, `btConeTwistConstraint`, `btSliderConstraint` e `btGeneric6DOFConstraint`.
- `btDiscreteDynamicsWorld` é o `UserCollisionAlgorithm` `btCollisionWorld`, e é um contêiner para corpos rígidos e restrições. Ele fornece a `stepSimulation` para prosseguir.
- `btMultiBody` é uma representação alternativa de uma hierarquia de corpos rígidos usando coordenadas generalizadas (ou reduzidas), usando o algoritmo do corpo articulado, conforme discutido por Roy Featherstone. A hierarquia da árvore começa com uma base fixa ou flutuante e corpos filhos, também chamados de links, são conectados por juntas: junta revoluta 1-DOF (semelhante à `btHingeConstraint` for `btRigidBody`), junta prismática 1-DOF (semelhante a `btSliderConstraint`). Note que `btMultiBody` foi introduzido no SDK de Bullet Physics relativamente recentemente e ainda está em desenvolvimento. Neste documento, apenas as coordenadas maximais baseadas em `btRigidBody` e `btTypedConstraints` são discutidas. Em uma revisão futura, um capítulo sobre `btMultiBody` será adicionado. Por enquanto, se você se interessa por `btMultiBody`, por favor veja o browser de exemplo e seu código-fonte em `examples/MultiBody` e `examples/ImportURDF`.

Corpos rígidos estáticos, dinâmicos e cinemáticos

Existem 3 tipos diferentes de objetos em Bullet:

- Corpos rígidos dinâmicos (em movimento)! massa positiva! em cada frame de simulação, a dinâmica atualizará sua transformação do mundo
- Corpos rígidos estáticos! massa zero

! não consigo me mover, só colidem

- Corpos rígidos cinemáticos!massa zero! podem ser animados pelo usuário, mas haverá interação unidirecional: objetos dinâmicos serão empurrados para longe, mas não há influência dos objetos dinâmicos

Todos eles precisam ser adicionados ao mundo da dinâmica. O corpo rígido pode receber uma forma de colisão. Essa forma pode ser usada para calcular a distribuição da massa, também chamada de tensor de inércia.

Transformação do Mundo do Centro de Massa

A transformada de mundo de um corpo rígido é, em Bullet, sempre igual ao seu centro de massa, e sua base também define seu referencial local para inércia. O tensor de inércia local depende da forma, e a classe `thebtCollisionShape` fornece um método para calcular a inércia local, dada uma massa.

Essa transformação de mundo precisa ser uma transformada de corpo rígido, o que significa que não deve conter escalamento, cisalhamento, etc. Se você quiser que um objeto seja escalado, pode escalar a forma de colisão. Outras transformações, como o cisalhamento, podem ser aplicadas (cozidas fornecidas) nos vértices de uma malha triangular, se necessário.

Caso a forma da colisão não esteja alinhada com a transformada do centro de massa, ela pode ser deslocada para corresponder. Para isso, você pode usar um `btCompoundShape` e usar a transformação filho para deslocar a forma de colisão filha.

O que é um MotionState?

MotionStates são uma forma do Bullet fazer todo o trabalho duro para você, transformando o mundo dos objetos simulados na parte de renderização do seu programa. Na maioria das situações, seu loop de jogo iteraria por todos os objetos que você está simulando antes de cada render. Para cada objeto, você atualizaria a posição do objeto de renderização a partir do `physicsbody`. Bullet usa algo chamado MotionStates para poupar esse esforço. Existem múltiplos outros benefícios do MotionStates:

- O cálculo envolvido no movimento de corpos é feito apenas para corpos que já se moveram; Não adianta atualizar a posição de um objeto de renderização a cada frame se ele não estiver se movendo.
- Você não precisa apenas renderizar coisas neles. Eles podem ser eficazes para notificar o código da rede de que um órgão se moveu e precisa ser atualizado em toda a rede.
- Interpolação geralmente só é significativa no contexto de algo visível na tela. Bullet gerencia a interpolação corporal através do MotionState.

- Você pode acompanhar uma mudança entre o objeto gráfico e a transformada do centro de massa.
- São fáceis

Interpolação

Bullet sabe como interpolar o movimento do corpo para você. Como mencionado, a implementação da interpolação é feita através do MotionStates. Se você tentar pedir a um corpo sua posição através do `btCollisionObject::getWorldTransform` or `btRigidBody::getCenterOfMassTransform`, ele retornará a posição no final do último tick físico. Isso é útil para muitas coisas, mas para renderização você vai precisar de alguma interpolação. Bullet interpola a transformação do corpo antes de passar o valor para `setWorldTransform`. Se você quiser a posição não interpolada de um corpo [que será a posição conforme calculada no final do último tick de física], use `btRigidBody::getWorldTransform()` e consulte o corpo diretamente.

Então, como eu uso um?

MotionStates são usados em dois lugares no Bullet. O primeiro é quando o corpo é criado pela primeira vez. Bullet pega a posição inicial do corpo a partir do estado de movimento quando o corpo entra na simulação. Bullet chama `getWorldTransform` com referência à variável que deseja que você preencha com `transforminformation`. Bullet também chama `getWorldTransform` em corpos cinemáticos. Por favor, veja a seção abaixo. Após a primeira atualização, durante a simulação Bullet chamará o estado de movimento para um corpo mover esse corpo. Bullet chama `setWorldTransform` com a transformação do corpo, para que você possa atualizar seu objeto. Adequadamente Para implementar um, basta herdar `btMotionState` e sobrescrever `getWorldTransform` e `setWorldTransform`.

DefaultMotionState

Embora recomendado, não é necessário derivar seu próprio estado de movimento a partir do `btMotionStateinterface`. O Bullet fornece um estado de movimento padrão que você pode usar para isso. Basta construir com a transformação padrão do seu corpo: `btDefaultMotionState* ms = new btDefaultMotionState();` Há um exemplo de um Estado de Movimento Ogre3D em um Apêndice.

Corpos Cinemáticos

Se você planeja animar ou mover objetos estáticos, deve marcá-los como cinemáticos. Também desative o sono/desativação deles durante a animação. Isso significa que o mundo da Bullet Dynamics receberá o novo worldtransform a partir do btMotionState a cada quadro de simulação.

```
body->setCollisionFlags( body->getCollisionFlags()  
|btCollisionObject::CF_KINEMATIC_OBJECT); corpo-  
>setAtivaçãoState(DISABLE_DEACTIVATION); Se você está usando corpos cinemáticos,  
então getWorldTransform é chamado de cada etapa de simulação. Isso significa que  
o estado de movimento do seu corpo cinemático deve ter um mecanismo para  
empurrar a posição atual do corpo cinemático para o estado de movimento.
```

Quadros de simulação e quadros de interpolação

Por padrão, a simulação de física de balas roda a uma taxa de quadros interna fixa de 60 Hertz (0,01666). O jogo ou aplicativo pode ter uma taxa de quadros diferente ou até variável. Para desacoplar a taxa de quadros de aplicação da taxa de quadros da simulação, um método automático de interpolação é incorporado à simulação a passos: quando o tempo delta da aplicação é menor que o passo fixo interno, o Bullet interpola a transformação do mundo e envia a transformação interpolada para o BtMotionState, sem realizar simulação física. Se o passo de tempo da aplicação for maior que 60 hertz, mais de 1 passo de simulação pode ser realizado durante cada chamada 'stepSimulation'. O usuário pode limitar o número máximo de etapas da simulação passando um valor máximo como segundo argumento.

Quando os rigidbodies forem criados, eles recuperarão a transformação inicial do mundo do btMotionState, usando btMotionState::getWorldTransform. Quando a simulação está rodando, usando o Simulação Step, a nova transformação do mundo é atualizada para corpos rígidos ativos usando thebtMotionState::setWorldTransform.

Corpos rígidos dinâmicos possuem massa positiva, e seu movimento é determinado pela simulação. Corpos rígidos estáticos e cinemáticos têm massa zero. Objetos estáticos nunca devem ser movidos pelo usuário.

7 Restrições

Existem várias restrições implementadas no Bullet. Veja exemplos/ConstraintDemo para um exemplo de cada um deles. Todas as restrições, incluindo o `btRaycastVehicle`, são derivadas do `btTypedConstraint`. Constraint atuam entre dois corpos rígidos, onde pelo menos um deles precisa ser dinâmico.

Restrição de Ponto a Ponto

A restrição ponto a ponto limita a translação, de modo que os pontos de pivô locais de 2 corpos rígidos correspondam no espaço do mundo. Uma cadeia de corpos rígidos pode ser conectada usando essa restrição.

```
btPoint2PointConstraint(btRigidBody& rbA, const btVector3& pivotInA); btPoint2PointConstraint(btRigidBody& rbA, btRigidBody& rbB, const btVector3& pivotInA, const btVector3& pivotInB);
```

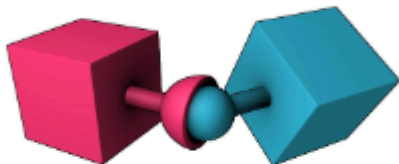


Figura 3 Restrição ponto a ponto

Restrição da Dobradiça

A restrição da dobradiça, ou junta revoluta, restringe dois graus angulares adicionais de liberdade, de modo que o corpo só pode girar em torno de um eixo, o eixo da dobradiça. Isso pode ser útil para representar portas ou rodas girando em torno de um eixo. O usuário pode especificar limites e motor para a dobradiça.

```
btHingeConstraint(btRigidBody& rbA, const btTransform& rbAFrame, bool useReferenceFrameA = false);  
btHingeConstraint(btRigidBody& rbA, const btVector3& pivotInA, btVector3& axisInA, bool useReferenceFrameA = false);  
btHingeConstraint(btRigidBody& rbA, btRigidBody& rbB, const btVector3& pivotInA, const btVector3& pivotInB, btVector3& axisInA, btVector3& axisInB, bool useReferenceFrameA = false); btHingeConstraint(btRigidBody& rbA, btRigidBody& rbB, const btTransform& rbAFrame, const btTransform& rbBFrame, bool useReferenceFrameA = false);
```



Figura 4 Restrição da Dobradiça

Restrição do Slider A restrição do slider permite que o corpo gire em torno de um eixo e translate ao longo desse eixo.

```
btSliderConstraint(btRigidBody& rbA, btRigidBody& rbB, const btTransform& frameInA, const  
btTransform& frameInB, bool useLinearReferenceFrameA);
```



Figura 5 Restrição do Deslizante

Restrição de Torção do Cone Para criar ragdolls, a restrição de torção cone é muito útil para membros como o braço superior. É uma restrição ponto a ponto especial que adiciona limites de cone e eixo de torção. O eixo x serve como eixo de torção.

```
btConeTwistConstraint(btRigidBody& rbA, const btTransform& rbAFrame);  
btConeTwistConstraint(btRigidBody& rbA, btRigidBody& rbB, const btTransform& rbAFrame, const btTransform& rbBFrame);
```

Restrição genérica de 6 Dof

Essa restrição genérica pode emular uma variedade de restrições padrão, configurando cada um dos 6 graus de liberdade (dof). Os primeiros 3 eixos dof são de eixo linear, que representam a translação de corpos rígidos, e os últimos eixos de 3 dof representam o movimento angular. Cada eixo pode ser travado, livre ou limitado. Na construção de um novo `btGeneric6DofSpring2Constraint`, todos os eixos são travados. Depois, o eixo pode ser reconfigurado. Note que várias combinações que incluem graus angulares livres e/ou limitados são indefinidas. Veja o `Bullet/exemplos/Dof6SpringSetup.cpp`.

```
btGeneric6DofConstraint(btRigidBody& rbA, btRigidBody& rbB, const btTransform& frameInA, const  
btTransform& frameInB, bool useLinearReferenceFrameA);
```

A seguir está a convenção:

```
btVector3 lowerSliderLimit = btVector3(-10,0,0);  
btVector3 hiSliderLimit = btVector3(10,0,0);  
  
slider btGeneric6DofSpring2Constraint* = novo  
btGeneric6DofSpring2Constraint(*d6body0,*fixedBody1,frameInA,frameInB); slider-  
>setLinearLowerLimit(lowerSliderLimit); slider->setLinearUpperLimit(hiSliderLimit);
```


Para cada eixo:

- Limite inferior == Limite superior -->  eixo está travado.
 - Limite inferior > Limite superior ---> eixo é livre
 - Limite inferior < Limite superior -->  eixo limitado nessa faixa
- Recomenda-se usar a restrição `btGeneric6DofSpring2Constraint`, que apresenta algumas melhorias em relação à restrição original `btGeneric6Dof(Spring)Constraint`.

8 Ações: Veículos e Controlador de Personagem

Interface de Ação

Em certos casos, é útil processar algum código personalizado de jogos de física dentro do pipeline de física. Embora seja possível usar um callback por tick, quando há vários objetos a serem atualizados, pode ser mais conveniente derivar sua classe personalizada a partir do `btActionInterface`. E implementar `thebtActionInterface::updateAction(btCollisionWorld* mundo, btScalar deltaTime)`; Existem exemplos embutidos, `btRaycastVehicle` e `btKinematicCharacterController`, que estão usando esse `btActionInterface`.

Veículo Raycast

Para simulações de veículos no estilo arcade, recomenda-se usar o modelo simplificado de veículo Bullet conforme fornecido no `btRaycastVehicle`. Em vez de simular cada roda e chassi como corpos rígidos separados, conectados por restrições, ele usa um modelo simplificado. Esse modelo simplificado traz muitos benefícios e é amplamente utilizado em jogos comerciais de corrida. O veículo inteiro é representado como um único corpo rígido, o chassi. A detecção de colisão das rodas é aproximada por raios, e o atrito dos pneus é um modelo básico de atrito anisotrópico. Veja `src/BulletDynamics/Vehicle` e `exemplos/ForkLiftDemo` para mais detalhes, ou confira os fóruns do Bullet. Kester Maddock compartilhou aqui um documento interessante sobre a simulação do veículo Bullet:

<http://tinyurl.com/ydfb7lm>

Controlador de Personagem

Um jogador ou personagem NPC pode ser construído usando uma forma de cápsula, esfera ou outra forma. Para evitar rotação, você pode definir o 'fator angular' a zero, o que desativa o efeito de rotação angular durante colisões e outras restrições. Veja `btRigidBody::setAngularFactor`. Outras opções (que não são recomendadas) incluem definir o tensor de inércia inversa a zero para o eixo de cima, ou usar a restrição de dobradiça apenas angular. Há também um `experimental3` `btKinematicCharacterController` como exemplo, um controlador não-physicalcharacter. Ele usa um `btGhostShape` para realizar consultas de colisão e criar um personagem capaz de subir escadas, deslizar suavemente pelas paredes, etc. Veja `src/BulletDynamics/Character` e `Demos/CharacterDemo` para seu uso.

3 `btKinematicCharacterController` tem vários problemas pendentes.

9 Dinâmica do Corpo Mole

Documentação preliminar

Introdução

A dinâmica de corpo macio fornece simulação de cordas, tecidos e corpos macios volumétricos, além da dinâmica de corpos rígidos existentes. Há interação bidirecional entre corpos moles, corpos rígidos e objetos de colisão.

- `btSoftBody` é o principal objeto de corpo mole. Ele é derivado de `btCollisionObject`. Ao contrário dos corpos rígidos, corpos moles não possuem uma única transformada de mundo: cada nó/vértice é especificado em coordenadas de mundo.
- `btSoftRigidDynamicsWorld` é o contêiner para corpos moles, corpos rígidos e objetos de colisão. É melhor aprender com exemplos/`SoftBodyDemo` como usar a simulação de corpos moles. Aqui estão algumas orientações básicas resumidamente:

Construção a partir de uma malha triangular

O `btSoftBodyHelpers::CreateFromTriMesh` pode criar automaticamente um corpo macio a partir de uma malha triangular.

Clusters de colisão

Por padrão, corpos suaves realizam detecção de colisão usando entre vértices (nós) e triângulos (faces). Isso requer uma tesselação densa, caso contrário, colisões podem ser perdidas. Um método aprimorado utiliza a decomposição automática em aglomerados convexos deformáveis. Para habilitar clusters de colisão, use:

```
psb->generateClusters(numSubdivisions);  
  
permitir colisão de cluster entre corpo macio e corpo rígido  
  
psb->m_cfg.collisions += btSoftBody::fCollision::CL_RS;  
  
Permitir colisão de cluster entre corpo macio e corpo macio  
  
psb->m_cfg.colisões += btSoftBody::fCollision::CL_SS;
```

O `Softbody` do `ExampleBrowser` possui uma opção de depuração para visualizar os clusters de colisão convexos.

Aplicando forças a um corpo macio

Existem métodos para aplicar uma força a cada vértice (nó) ou a um nó individual:

```
softbody ->addForce(const btVector3& forceVector);  
softbody ->addForce(const btVector3& forceVector,nó int);
```

Restrições de corpo mole

É possível fixar um ou mais vértices (nós), tornando-o inamovível:

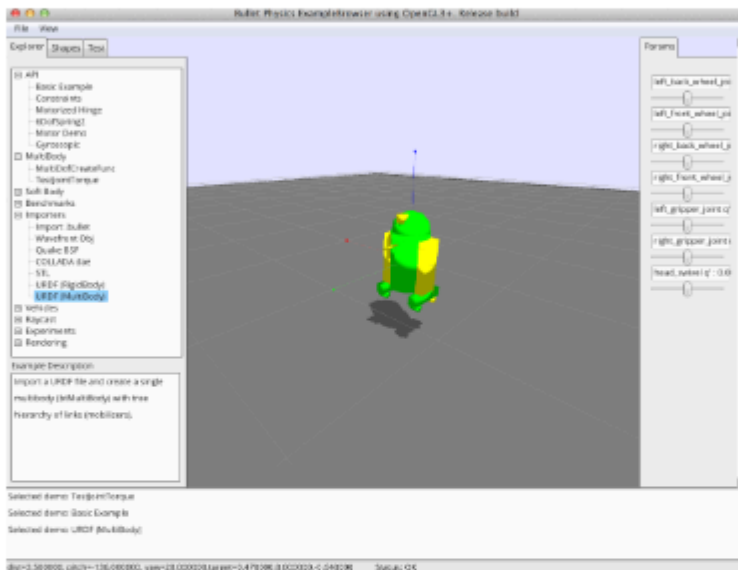
```
softbody->setMass(node,0.f);
```

ou para anexar um ou mais vértices de um corpo mole a um corpo rígido:

```
softbody->appendAnchor(int node,btRigidBody* rigidbody,  
booldisableCollisionThweenLinkedBodies=false); Também é possível fixar  
dois corpos macios usando restrições, veja Bullet/Demos/SoftBody.
```

Navegador de Exemplos de 10 Pontos

O Bullet 2.83 introduz um novo navegador de exemplo baseado no OpenGL 3+, substituindo o anterior Glutdemos. Existe uma opção de linha de comando com suporte limitado para OpenGL 2: -opengl2O navegador de exemplo é testado no Windows, Linux e Mac OSX. Ele tem suporte a Retina, então fica melhor no Mac OSX. Aqui está uma captura de tela:



Cada exemplo também pode ser compilado de forma independente, sem gráficos. Veja theexamples/BasicDemo/main.cpp como fazer isso.

BSP Demo

Importar arquivos .bsp do Quake e converter os pincéis em objetos convexos. Isso funciona melhor do que usar triângulos.

Demonstração de Veículos

Esta demonstração mostra o uso do veículo embutido. As rodas são aproximadas por raios de fundição. Essa aproximação funciona muito bem para veículos em movimento rápido.

Demonstração com Empilhadeira

Uma demonstração que mostra como usar restrições como restrição de dobradiça e deslizante para construir um veículo empilhadeira.

11 Demonstrações Técnicas Avançadas de Baixo Nível

Demonstração de Interface de Colisão

Esta demonstração mostra como usar a detecção de colisão de balas sem a dinâmica. Ele usa a classe `thebtCollisionWorld` e preenche esse espaço com `btCollisionObjects`. O método `PerformDiscreteCollisionDetection` é chamado e a demonstração mostra como reunir os pontos de contato.

Demonstração de Colisão

Esta demo é mais de baixo nível do que a demonstração anterior de Interface de Colisões. Ele usa diretamente o `thebtGJKPairDetector` para consultar os pontos mais próximos entre dois objetos.

Algoritmo de Colisão do Usuário

Mostra como você pode registrar seu próprio algoritmo de detecção de colisão que gerencia a detecção de colisão para um determinado par de tipos de colisão. Um caso simples de esfera-esfera sobrepõe-se à detecção padrão GJK.

Elenco Convexo do GJK / Demo de Varredura

Esta demonstração mostra como realizar uma varredura linear entre dois objetos de colisão e retorna o tempo de impacto. Isso pode ser útil para evitar penetrações no controle da câmera e do personagem.

Colisão Convexa Contínua

Mostra consulta de tempo de impacto usando detecção contínua de colisão, entre dois objetos girando e translacionando. Ele utiliza a implementação do Progresso Conservador feita por Bullet.

Demonstração de Raytracer

Isso mostra o uso da fundição de raios CCD em formas de colisão. Ele implementa um ray tracer que pode visualizar com precisão a representação implícita das formas de colisão. Isso inclui a margem de colisão, invólucros convexos de objetos implícitos, somas de Minkowski e outras formas que seriam difíceis de visualizar de outra forma.

Simplex Demo

Esta é uma demonstração de nível muito baixo testando o funcionamento interno do algoritmo de subdistância GJK. Isso calcula a distância entre um simplex e a origem, que é desenhada com uma linha vermelha. Um simplex contém de 1 a 4 pontos, a demonstração mostra o caso de 4 pontos, um tetraedro. O solver de Voronoi simplex é usado, conforme descrito por Christer Ericson em seu livro de detecção de colisão.

12 Ferramentas de Autoria e Serialização

Formas de colisão, corpos rígidos e restrições podem ser criados em uma ferramenta de autoria 3D e exportados para um formato de arquivo que o Bullet pode ler.

Dynamica Maya Plugin

A Walt Disney Animation Studios contribuiu com seu plugin interno Maya para criar Bullet collisionshapes e rigid bodies como código aberto. Dynamica pode simular dinâmicas de corpo rígido dentro do Maya e pode exportar para arquivos de física de balas e COLLADA Physics. A versão mais recente tem suporte preliminar para tecido ou corpo macio. Há mais informações na página da wiki Bullet. Você pode baixar uma versão pré-compilada do plugin Dynamica para Windows ou Mac OSX de <http://bullet.googlecode.com>. The repositório de código-fonte do Dynamica está sob <http://dynamica.googlecode.com>

BlenderA suíte de produção 3D open source Blender usa física de balas para animações e seu motor de jogo interno. Veja <http://blender.org> Blender tem uma opção para exportar arquivos de física COLLADA. Há também um projeto que pode ler diretamente qualquer informação de um arquivo .blend do Blender, incluindo forma de colisão, corpo rígido e informação de restrições. See <http://gamekit.googlecode.com> Blender 2.57 e posteriores têm uma opção para exportar arquivos .bullet diretamente do motor do jogo. Isso pode ser feito usando o comando `Python exportBulletFile("name.bullet")` no módulo `PhysicsConstraints`.

Cinema 4D, Lightwave CORE, Houdini Cinema 4D 11.5 usa Bullet para a simulação de corpo rígido, e há um relato de que Lightwave CORE também planeja usar Bullet.

Para o Houdini existe um DOP/plugin,
see <http://code.google.com/p/bullet-physics-solver/>

Serialização e o formato binário Bullet .bullet

A partir do bullet 2.76 tem a capacidade de salvar o mundo da dinâmica em um dump binário. Salvar os objetos e formas em um buffer é integrado, então não são necessárias bibliotecas adicionais. Aqui está um exemplo de como salvar o mundo dinâmico em um arquivo .bullet binário:

```
btDefaultSerializer* serializer = new btDefaultSerializer();  
dynamicsWorld->serialize(serializer);  
arquivo FILE* = fopen("testFile.bullet", "wb");  
fwrite(serializer->getBufferPointer(), serializer->getCurrentBufferSize(), 1, arquivo);  
fclose(arquivo);
```

Você pode pressionar a tecla F3 na maioria dos exemplos de Bullet para salvar um 'testFile.bullet'. Você pode ler .bulletfiles usando o btBulletWorldImporter conforme implementado em theBullet/examples/Importers/ImportBullet. Mais informações sobre serialização .bullet estão na wiki do Bullet em

http://bulletphysics.org/mediawiki-1.5.8/index.php/Bullet_binary_serialization

13 Dicas Gerais

Evite formas de colisão muito pequenas e muito grandes

O tamanho mínimo de objeto para objetos em movimento é cerca de 0,2 unidades, sendo 20 centímetros para a gravidade da Terra. Se objetos menores ou gravidade maior forem manipulados, reduza a frequência interna da simulação de acordo, usando o terceiro argumento de `btDiscreteDynamicsWorld::stepSimulation`. Por padrão, é 60Hz. Por exemplo, simular um lançamento de dados (caixa de 1cm de largura com gravidade de 9,8m/s²) requer uma frequência de pelo menos 300Hz (1./300). Recomenda-se manter o tamanho máximo dos objetos em movimento menor que cerca de 5 unidades/metros.

Evite grandes razões de massa (diferenças)

A simulação torna-se instável quando um objeto pesado repousa sobre um objeto muito leve. É melhor manter a massa em torno de 1. Isso significa que a interação precisa entre um tanque e um objeto muito leve não é realista.

Combine múltiplas malhas triângulos estáticas em uma só

Muitos pequenos `btBvhTriangleMeshShape` poluem a `broadphase`. Melhor combinar os dois.

Use o passo de tempo fixo interno padrão

Bullet funciona melhor com um intervalo de tempo interno fixo de pelo menos 60 hertz (1/60 de segundo). Para segurança e estabilidade, o Bullet subdividirá automaticamente o passo de tempo variável em subpassos fixos de simulação interna, até um número máximo de subpassos especificado como segundo argumento para `passarSimulação`. Quando o passo de tempo é menor que o subpasso interno, Bullet interpola o movimento. Esse mecanismo de segurança pode ser desativado passando 0 como número máximo de subpassos (segundo argumento para `stepSimulation`): o passo de tempo interno e os subpassos são desativados, e o passo de tempo real é simulado. Não é recomendado desativar esse mecanismo de segurança.

Para ragdolls, use `btConeTwistConstraint`

É melhor construir um ragdoll com `btHingeConstraint` e/ou `btConeTwistLimit` para joelhos, cotovelos e braços.

Não defina a margem de colisão para zero

O sistema de detecção de colisão precisa de alguma margem para desempenho e estabilidade. Se a folga for perceptível, por favor, compense a representação gráfica.

Use menos de 100 vértices em uma malha convexa

É melhor manter limitado o número de vértices em um `btConvexHullShape`. É melhor para desempenho, e muitos vértices podem causar instabilidade. Use a utilidade `btShapeHull` para simplificar os invólucros convexos.

Evite triângulos enormes ou degenerados em uma malha triangular

Mantenha o tamanho dos triângulos razoável, digamos abaixo de 10 unidades/metros. Também é melhor evitar triângulos degenerados com grandes proporções de tamanho entre os lados ou área próxima a zero.

O recurso de perfilamento `btQuickProf` ignora o alocador de memória

Se necessário, desative o profiler ao verificar vazamentos de memória ou ao criar a versão final do seu lançamento de software. O recurso de perfilagem pode ser desativado definindo `#defineBT_NO_PROFILE 1` em `Bullet/src/LinearMath/btQuickProf.h`

Tópicos Avançados

Valor de atrito e restituição por triângulo

Por padrão, há apenas um valor de atrito para um corpo rígido. Você pode conseguir atrito por forma ou por triângulo para mais detalhes. Veja o `Demos/ConcaveDemo` sobre como definir o atrito por triângulo. Basicamente, adicione `CF_CUSTOM_MATERIAL_CALLBACK` às flags de colisão ou ao `rigidbody`, e registre uma função global de callback de material. Para identificar o triângulo na malha, tanto o `triangleID` quanto o `partID` da malha são passados para o callback do material. Isso corresponde ao `triangleID/partID` da interface `stridingmesh`. Uma maneira mais fácil é usar o `btMultimaterialTriangleMeshShape`. Veja `theDemos/MultiMaterialDemo` para uso.

Outros Solucionadores de Restrições MLCP

Bullet usa seu `btSequentialImpulseConstraintSolver` por padrão. Você pode usar um solucionador de restrições diferente, passando-o para o construtor do seu `btDynamicsWorld`. Esses solucionadores alternativos de restrições MLCP estão em `Bullet/src/BulletDynamics/MLCPSolvers`. Veja o código-fonte de `examples/vehicles/VehicleDemo` como usar um solucionador de restrições diferente.

Modelo de Atrito Personalizado

Se você quiser ter um modelo de atrito diferente para certos tipos de objetos, pode registrar uma função de atrito no solucionador de restrições para certos tipos de corpo. Esse recurso não é compatível com a configuração de solucionador de restrições amigável ao cache. Veja `#define USER_DEFINED_FRICTION_MODEL` em `Demos/CcdPhysicsDemo.cpp`.

14 Paralelismo usando OpenCL

OpenCL corpo rígido e detecção de colisão.

Implementamos do zero um pipeline de corpo rígido e detecção de colisão que roda 100% usando kernels OpenCL. Funciona melhor em GPUs discretas de alta qualidade como AMD 7970 ou mais recentes, ou NVIDIA GTX 680 ou mais recentes. Um exemplo simples do OpenCL está desativado por padrão no navegador de exemplo. Se você tiver o hardware da GPU adequado e o driver/compilador OpenCL atualizado, pode usar a seguinte opção de linha de comando:

--enable_experimental_openglNote que existem muitas razões pelas quais os kernels do OpenCL falham, e você precisará se familiarizar com o OpenCL para lidar com esses problemas. Por favor, veja o documento PDF separado com mais informações sobre a detecção de colisão do OpenCL e o pipeline de corpo rígido na pasta Bullet/docs. Há também um capítulo de livro sobre o pipeline de corpo rígido OpenCL como parte do livro 'Multithreading in Visual Effects' da CRC Press. Este livro também está disponível na Amazon.

15 Documentação e referências adicionais

Recursos online

Visite o site Bullet Physics em <http://bulletphysics.org> para um fórum de discussão, uma wiki com perguntas frequentes e dicas e download da versão mais recente. A página da Wikipédia lista alguns jogos e filmes usando Bullet at [http://en.wikipedia.org/wiki/Bullet_\(software\)](http://en.wikipedia.org/wiki/Bullet_(software))

Ferramentas de Autoria

- Suporte ao plugin Dynamica Maya e Bullet COLLADA Physics
at <http://dynamica.googlecode.com>
- O modelador 3D do Blender inclui suporte a física Bullet e COLLADA:
<http://www.blender.org>
- Padrão de física COLLADA: <http://www.khronos.org/collada>

Livros

- Detecção de Colisões em Tempo Real, Christer Ericson <http://www.realtimecollisiondetection.net/Bullet> usa o solucionador de voronoi simplex discutido para GJK
- Detecção de Colisão em Ambientes 3D Interativos, Gino van den também Bergen <http://www.dtecta.com> site para biblioteca sólida de detecção de colisão. Discute GJK e outros algoritmos, muito útil para entender o Bullet
- Animação baseada em física, Kenny Erleben <http://www.diku.dk/~kenny/> Very útil para entender a dinâmica e restrições de bullet
- Multithreading em Efeitos Visuais Discutiu o trabalho de corpo rígido OpenCL para Física de Balas.

Contribuições e pessoas

A biblioteca Bullet Physics está em desenvolvimento ativo em colaboração com muitos desenvolvedores profissionais de jogos, estúdios de cinema, além de academia, estudantes e entusiastas. O autor principal e líder do projeto é Erwin Coumans, que iniciou o projeto na Sony ComputerEntertainment America em P&D nos EUA, depois na Advanced Micro Devices e agora no Google. Algumas pessoas que contribuíram com código-fonte para Bullet: Roman Ponomarev, SCEA, restrições, pesquisa CUDA e OpenCL John McCutchan, SCEA, ray cast, controle de caracteres, várias melhorias Nathanael Presson, Havok: autor inicial de Bullet soft body dynamics e EPAG Ino van den Bergen, Dtect: LinearMath classes, várias ideias de detecção de colisãoChrister Ericson, SCEA: voronoi simplex solverPhil Knight, Disney Avalanche Studios: compatibilidade multiplataforma, serialização BVH Ole Kniemeyer, Maxon: vários patches gerais, btConvexHullComputer Simon Hobbs, SCEE: varredura e poda de eixos 3D e partes de btPolyhedralContactClippingPierre Terdiman, NVIDIA: vários trabalhos relacionados à separação de testes de eixos, varrimento e poda Dirk Gregorius, Factor 5: discussão e assistência com RestriçõesErin Catto, Blizzard: impulso acumulado em impulso sequencialFrancisco Leon: GIMPACT colisão côncava côncavaEric Sunshine: sistema de construção jam + msvcgen (substituído por cmake desde Bullet 2.76)Steve Baker: física da GPU e melhorias gerais na implementaçãoJay Lee, TrionWorld: suporte de precisão duplaKleMiX, também conhecido como Vsevolod Klementjev, versão gerenciada, port C# para XNAMarten Svanfeldt, Starbreeze: solucionador de restrições paralelas e outras melhorias e otimizaçõesMarcus Hennix, Starbreeze: btConeTwistConstaint etc. Arthur Shek, Nicola Candussi, Lawrence Chai, Disney Animation: Dynamica Maya Plugin Muitas outras pessoas contribuíram para o Bullet, obrigado a todos nos fóruns do Bullet.

